

Program Verification: Beyond Function Contracts

Journée LVP

Virgile Prevosto

November 14th, 2024

Université Paris-Saclay, CEA, List





Acknowledgments

Joint work with

Lionel Blatter, Sébastien Patte, Virgile Robles, Nicolas Stouls, Nikolaï Kosmatov, Pascale Le Gall, Valentin Perrelle
... and all Frama-C's contributors

Program Verification

Program

```
1  int main() {
2    status res = OK;
3    while(res==OK) {
4      get_input();
5      res = update_state();
6    }
7    return res;
8  }
```

Formal specification

Abstract interpretation
Deductive verification
Runtime verification
Model checking

✔No runtime error

✔Property 1

✔Property 2

✔…

How do we specify properties?

- Function contracts and assertions
- Temporal logic formulas

Function Contracts

```
1  /*@
2   requires \valid(state + (0 .. n - 1));
3   requires
4     \forall integer i;
5     0 <= i < n ==> elem_ok(state[i]);
6   assigns state[0 .. n];
7   ensures computation_ok(state);
8  */
9  void f(char* state, size_t n) {
10 ...
11 //@ assert some_property(state);
12 ...
13 }
```

- Pre-condition: what must be verified before calling `f`
- Post-condition: what `f` guarantees when it returns
- Specifies which locations `f` may assign to
- Assertions at specific program points

Position of the problem

- Tools verify that the code is conforming to its specification.
- How confident can we be that annotations are really what we want to express?
- In particular if they are big/numerous
  - Example from Thales[1]: ˜400kLoC of ACSL annotations (from ˜500 lines of MetAcsl).
- And/or if we need to encode part of the property with ghost code.

[1] Djoudi et al. Formal verification of a JavaCard virtual machine with Frama-C. FM 2021

Examples of Properties Beyond Function Contracts

What function contracts are good at

- Expressing a property that (should) hold at a specific program point
- or pre- and post- states for a contract
- or a few more within the same function with `\at`

More complex situations

- Check a property at a large number of program points
- Compare several executions of a function/some functions
- Check a sequence of events across the whole execution path
- Check sequence of operations on objects of a given type

- A Framework for modular analysis of C code.
- Released under LGPL (v30.0 Beta last week)
- Extensible platform



- A Framework for modular analysis of C code.
- Released under LGPL (v30.0 Beta last week)
- Extensible platform



Specification and Verification Process

- Dedicated Specification Language
- Translation into ACSL (+ ghost code)
- Dedicated Verification
- Formalization



MetAcsl at a glance

- Developed by Virgile Robles during his PhD.
- Open-source (LGPL): `https://git.frama-c.com/pub/meta/-/releases`
- HIgh-Level ACSL REquirement.
- Properties that must be verified at various program points (context).
- Meta-variables depending on the context.
- Each HILARE is instantiated as (many) ACSL annotations.

Example of HILARE

```
meta \prop, \name(confidential_read),
  \targets(\diff(\ALL, init)), \context(\reading),
  \forall int p;
    0 <= p < PAGE_NB && pages[p].status == ALLOCATED &&
     user_level < pages[p].level
    ==> \separated(\read, pages[p].data + (0 .. PAGE_SIZE - 1));

meta \prop, \name(constant_conf_level),
  \targets(\diff(\ALL, {init, page_encrypt, page_decrypt})),
  \context(\writing),
  \forall int p; 0 <= p < PAGE_NB && pages[p].status == ALLOCATED ==>
   \separated(\written,pages[p].level);
```

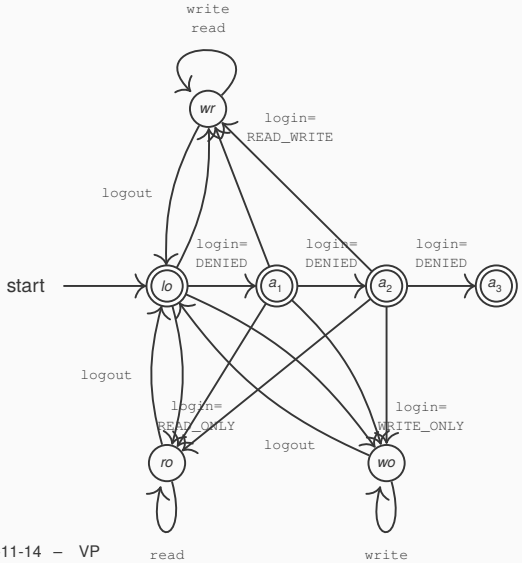
Relational Properties: Rpp

- Developped by Lionel Blatter during his PhD
- Express relational properties between function calls

Non-interference

```
1  extern int private, public;
2
3  //@ assigns public,private \from public,private;
4  void f() {
5    if (public < 1000) public++;
6    if (private > public) private-=public;
7  }
8
9  /*@ relational
10     \callset(\call(f,c1),\call(f,c2)) ==>
11     \at(public,Pre_c1) == \at(public,Pre_c2)
12     ==>
13     \at(public,Post_c1) == \at(public,Post_c2);
14 */
```

Automata: Aoraï



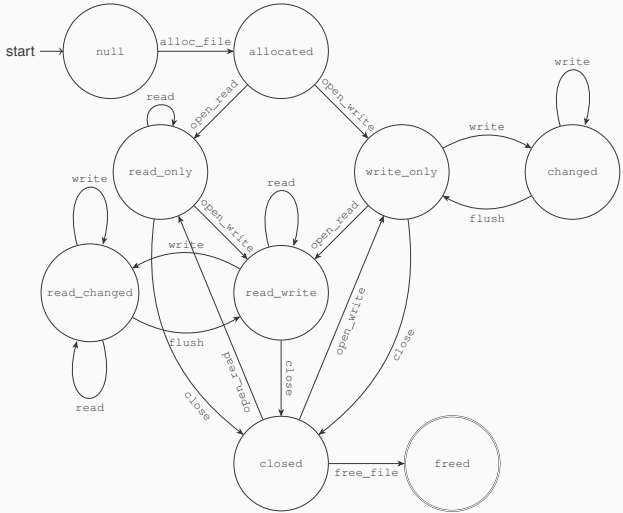
- Initial version by Nicolas Stouls (Inria/INSA Lyon).
- Automaton encoding admissible call sequences during an execution of the program

Automaton in Aoraï

```
1  logged_out: { login() {{ \result == -1 }} } -> attempt_1
2            | { login() {{ \result == 0 }} } -> logged_ro
3            | { login() {{ \result == 1 }} } -> logged_wo
4            | { login() {{ \result == 2 }} } -> logged_rw;
5
6  attempt_1:  { login() {{ \result == -1 }} } -> attempt_2
7            | { login() {{ \result == 0 }} } -> logged_ro
8            | { login() {{ \result == 1 }} } -> logged_wo
9            | { login() {{ \result == 2 }} } -> logged_rw;
10
11 attempt_2:  { login() {{ \result == -1 }} } -> too_many_attempts
12           | { login() {{ \result == 0 }} } -> logged_ro
13           | { login() {{ \result == 1 }} } -> logged_wo
14           | { login() {{ \result == 2 }} } -> logged_rw;
15 ...
```

Typestates[1]

- Refinement on the type of an object
- Typestate varies during lifetime
- Admissible operations depend on typestate (and change it)
- Plug-in (unreleased yet) by Sébastien Patte



[1] Strom and Yemini. IEEE Trans. on Soft. Eng., 12:1. 1986

Typestates Plug-in Syntax

```
1  struct inner_repr;
2  typedef struct inner_repr *file;
3
4  file alloc_file(const char* name);
5
6  void open_read(file f);
7
8  void open_write(file f);
9
10 char read(file f);
11
12 void write(file f, char c);
13
14 void flush(file f);
15
16 void close(file f);
17
18 void free_file(file f);
```

```
1  /*@ typestate \def,
2       (struct inner repr) f =
3       null | allocated | read_only
4       | ...
5  */
6
7  /*@ typestate \trans,
8        {alloc_file},
9        \result==allocated;
10 */
11
12 /*@ typestate \trans,
13       f==allocated,
14       {open_read},
15       f==read_only;
16 */
17
18 ...
```

Specification and Verification Process

- Dedicated Specification Language
- Translation into ACSL (+ ghost code)
- Dedicated Verification
- Formalization

Generation of HILAREs

```c
int x, y;
/*@ assigns x \from n; */
extern void set_x(int n);

/*@
meta \prop,\name(example),
\targets(\diff(\ALL,set_x)),
\context(\writing),
\separated(\written,&x);
*/

void test(int* p) {
  *p = 1;
  y = 2;
}
```

```
frama-c -meta meta_test.c
```

```c
void test(int *p)
{
  /*@ assert example:  meta:
        \separated(p, &x); */
  *p = 1;
  y = 2;
  return;
}
```

- Some (trivial) simplifications are done

Use standard analysis plug-ins

```
frama-c -meta meta_test.c -then-last -wp
```

- can also work with Eva or E-ACSL
- Wp may require additional annotations
- Some of which might come from other HILAREs

```
frama-c -rpp rpp_test.c -then -wp
```

Instrumented code

```
1  void relational_wrapper_2(void)
2  {
3    if (public_c1_2 < 1000) public_c1_2 ++;
4    if (private_c1_2 > public_c1_2) private_c1_2 -= public_c1_2;
5    ;
6    if (public_c2_2 < 1000) public_c2_2 ++;
7    if (private_c2_2 > public_c2_2) private_c2_2 -= public_c2_2;
8    ;
9    /*@ check Rpp:
10         \at(public_c1_2,Pre) == \at(public_c2_2,Pre) ==>
11         \at(public_c1_2,Here) == \at(public_c2_2,Here);
12   */
13   return;
14 }
```

```
frama-c -aorai-automata automaton.ya test_aorai.c -then-last -eva
```

Ghost functions advancing automaton

```
1  /*@ ghost void login_post_func(enum permission res) {
2   /@ slevel full; @/
3   ...
4   if (aorai_intermediate_state_7 == 1)
5    if (res == -1) too_many_attempts_tmp = 1; else too_many_attempts_tmp = 0;
6   else too_many_attempts_tmp = 0;
7    if (reject == 1) reject_tmp = 1; else reject_tmp = 0;
8     if (aorai_intermediate_state_1 == 1) {
9      if (res == 1) logged_wo_tmp = 1; else goto __aorai_label_16;
10    }
11    else
12    ...
13 } */
```

Aoraï and Eva

- Aoraï's properties intrinsically a whole-program analysis
- Eva or E-ACSL are the primary candidates for verification
- Eva-related options and built-ins:
  - `Frama_C_show_aorai_state`
  - `-aorai-instrumentation-history`

Typestates Instrumented Code

- Each variable of relevant type associated to a (ghost) tracker
- Contracts for the function of the API
- Special treatment for allocating functions

```
1  enum ts_inner_repr { null = 0, ... }
2
3  /*@ requires \valid(ts_f);
4      requires *ts_f == allocated
5             || *ts_f == read_only
6             || *ts_f == closed;
7      behavior b1:
8      assumes *ts_f == allocated;
9      ensures *ts_f == write_only;
10 */
11 open_write(file f)
12   /*@ ghost (enum ts_inner_repr \ghost *ts_f) */;
13
14 int main() {
15   file test = alloc_file("test");
16   /*@ ghost enum ts_inner_repr \ghost*
17        ts_test = ts_alloc_file(); */
18   open_write(test)/*@ ghost (ts_test) */;
19 }
```

Specification and Verification Process

- Dedicated Specification Language
- Translation into ACSL (+ ghost code)
- Dedicated Verification
- Formalization

Position of the problem

- Each HILARE can generate many ACSL annotations
- For Wp, local contexts often need to be enriched with auxiliary annotations
- What if we could deduce an HILARE from other ones directly at high-level
  - at least for some deduction patterns

Initial Motivation

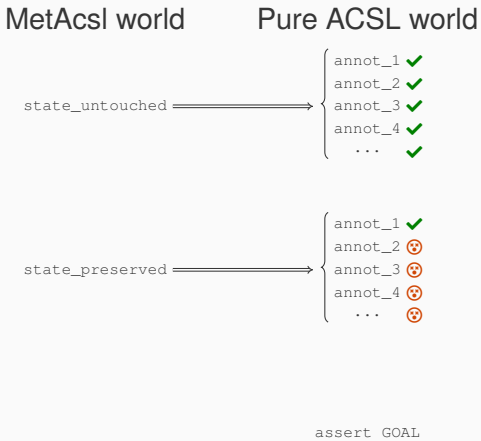
```
1  /*@
2   ensures state == 42;
3  */
4  void transition () {
5   // a new state is computed
6   ...
7   //@ assert KNOWN : state == 42;
8   util ();
9   //calling utility functions
10  //does preserve the state value
11  //@ assert GOAL : state == 42;
12 }
```

- `util` performs many computations, irrelevant to `state`
- and has (transitively) many callees
- Traditional Wp approach would require many annotations, mostly irrelevant

Auxiliary HILAREs

```
1  /*@ meta \prop ,
2    \name(state_untouched),
3    \targets(\callees(util)),
4    \context(\writing),
5    \separated(\written,&state); */
```

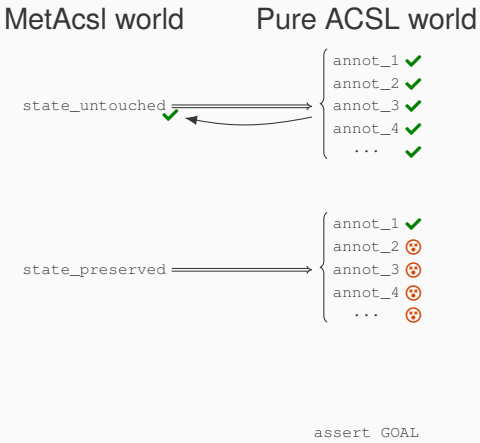
```
1  /*@ meta \prop,
2    \name(state_preserved),
3    \targets(\callees(util)),
4    \context(\postcond),
5    state == \old(state); */
```



Auxiliary HILAREs

```
1  /*@ meta \prop ,
2    \name(state_untouched),
3    \targets(\callees(util)),
4    \context(\writing),
5    \separated(\written,&state); */
```

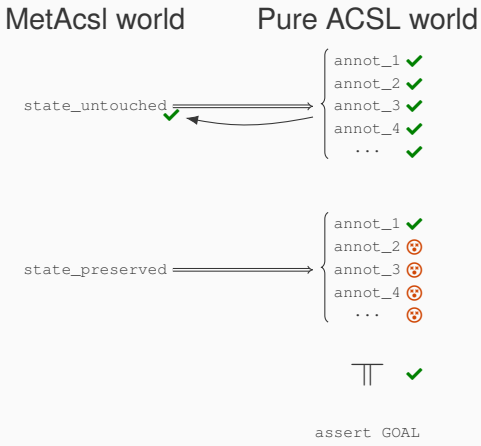
```
1  /*@ meta \prop,
2    \name(state_preserved),
3    \targets(\callees(util)),
4    \context(\postcond),
5    state == \old(state); */
```



Auxiliary HILAREs

```
1  /*@ meta \prop ,
2    \name(state_untouched),
3    \targets(\callees(util)),
4    \context(\writing),
5    \separated(\written,&state); */
```

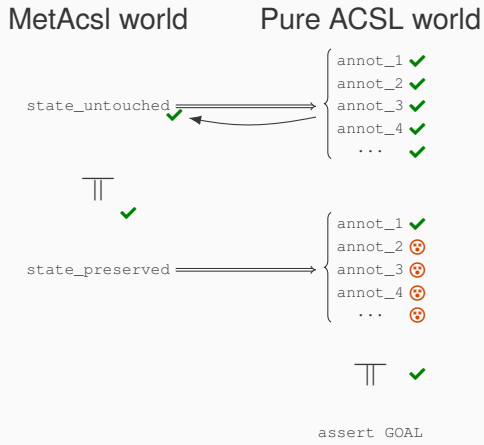
```
1  /*@ meta \prop,
2    \name(state_preserved),
3    \targets(\callees(util)),
4    \context(\postcond),
5    state == \old(state); */
```

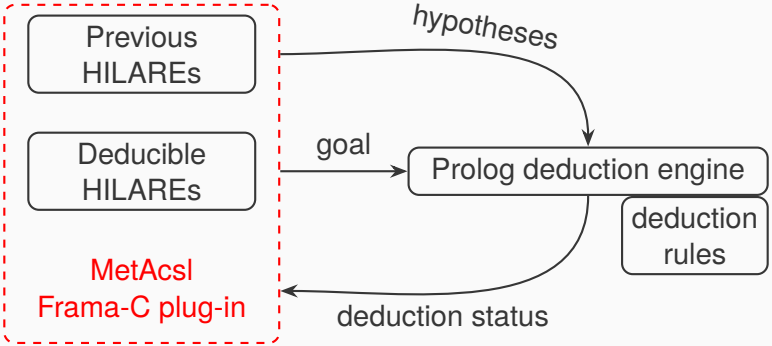


Auxiliary HILAREs

```
1  /*@ meta \prop ,
2    \name(state_untouched),
3    \targets(\callees(util)),
4    \context(\writing),
5    \separated(\written,&state); */
```

```
1  /*@ meta \prop,
2    \name(state_preserved),
3    \targets(\callees(util)),
4    \context(\postcond),
5    state == \old(state); */
```




Previous
HILAREs
Deducible
HILAREs
MetAcsl
Frama-C plug-in
hypotheses
goal
Prolog deduction engine
deduction
rules
deduction status

Prolog Deduction Engine

- Use SWIProlog and {log}
- “Basic” layer encoding our pattern
- “Intermediate” layer of generic facts

```
meta_valid("Postcond",
           negative_assigns(L), S)
:-
     meta_inter("Writing",
                not_written(L), S)
    &
    callees_restrict(S, S).

% ...

meta_inter(C, P, S) :-
   subsumes(C, P, S, _).
```

From MetAcsl to Prolog

```
1  targets({f_transition,
2           f_util,
3           f_util_callee_1, ...}).
4
5  calls(f_transition, f_util).
6  calls(f_util, f_util_callee_1).
7  ...
8
9  meta_ground("Writing",
10   not_written(state),
11   {f_util, f_util_callee_1, ...}).
12
13 go :-
14  meta_valid(
15   "Postcond",
16   negative_assigns(state),
17   {f_util, f_util_callee_1, ...}).
```

Encoding basic facts about the program

- Call graph
- HILAREs previously proved
- Goal that must be discharged

Verifying Relational Properties Beyond Self-composition

- Transformation becomes very complex in presence of pointers (and aliasing)
- More flexible solution using directly Wp primitives
- But not implemented in the Rpp plug-in itself

Single Function

$$\forall \sigma, \sigma', P(\sigma) \Rightarrow WP(S, \sigma, \sigma', Q(\sigma, \sigma'))$$

Relational Case

$$\forall \sigma_1, \sigma_1', \sigma_2, \sigma_2', P(\sigma_1, \sigma_2) \Rightarrow WP(S_1, \sigma_1, \sigma_1', WP(S_2, \sigma_2, \sigma_2', Q(\sigma_1, \sigma_1', \sigma_2, \sigma_2')))$$

Iterate process for more than 2 functions

Specification and Verification Process

- Dedicated Specification Language
- Translation into ACSL (+ ghost code)
- Dedicated Verification
- Formalization

MetAcsl's Why3 Model

A very crude model

```
type instruction =
| Read location
| Write location
| Call function_name

type block = L.list instruction

type func = { def: block }

type program =
  M.fmap function_name func
```

But sufficient to prove our patterns

```
lemma negative_assigns:
 forall prog, l, f, cs,
        pre, post, v.
  valid_program prog ->
  callees prog f cs ->
  untouched f l ->
  (forall f'.
  S.mem f' cs -> untouched f' l) ->
  state_block prog f pre post ->
  (M.mapsto l v pre
    <-> M.mapsto l v post)
```

Formalizing a Relational VCGen

- Done in Coq [2]
- Small imperative language with function calls and pointer arithmetic
- Complete formalization of an optimized VCGen
- ... and its relational extension

[2] Blatter et al., *An Efficient VCGen-Based Modular Verification of Relational Properties*. ISoLA'22, LNCS 13701

Typestates Formalization

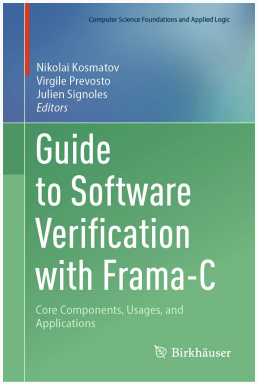
- Work in progress
- Tiny language with pointers and function calls
- Semantics defined in Skel[3]
- Instrumentation function as well
- Proofs in Coq through `necrocoq` tool

[3] https://skeletons.inria.fr/

Conclusion

- Poweful plug-ins to specify various kinds of properties
- Verification by direct translation into ACSL
- Dedicated verification techniques
- Good example of plug-in collaboration within Frama-C
- More information in Chap. 10 of the Frama-C Book
- Other languages not mentioned here: CSpeL[a], CaFE[b], SecureFlow[c], ...

[a] Myriam Clouet's PhD
[b] Steven de Oliveira's PhD
[c] Mounir Assaf's PhD



Perspectives - Specification

- Extend existing DSL (more HILARE contexts, new meta-variables, refine Aoraï automata, ...)
- Mixing properties (HILARE + Aoraï or HILARE + typestate)
- Multi-language specifications, and FFI specification

Perspectives - Verification

- Model Checking[4]
- More meta-deduction rules for MetAcsl
- Specific Eva domains for Aoraï and Typestates
- Refined Relational VCGen (with relational invariants)

[4] Amilon and Gurov. Deductively Verified Program Models for Software Model Checking. ISoLA 2024

Perspectives - Formalization

- Finer grained model for MetAcsl
- More realistic language for Typestates
- Towards a full ACSL formalization[5]

[5] Louis Gauthier's PhD